

Tema 5 — Colecciones de objetos

Laboratorio Guiado

Objetivos	Practicar el uso de colecciones en Python: listas, tuplas, conjuntos y diccionarios; recorrerlas, modificarlas, filtrarlas, anidarlas y combinarlas con objetos sencillos.
Material necesario	Terminal, Python 3.13, venv del Tema05, Visual Studio Code con extensiones Python y Jupyter, scripts del tema instalados en ~/Curso_Python/Tema05.

Laboratorio 1. Listas

1. Actualizar el repositorio y abrir la carpeta del tema y activar el entorno virtual. El venv ya está creado.

```
git pull origin main
cd ~/Curso_Python/Tema05
source .venv/bin/activate
python --version
code .
```

2. Abrir el fichero 01_listas.py. Revisar la creación de listas, el acceso por índice y los índices negativos.

```
servicios = ["ssh", "nginx", "postgresql"]
puertos = [22, 80, 5432]

print("Servicios:", servicios)
print("Tipo:", type(servicios))
print("Número de servicios:", len(servicios))
print("Primer servicio:", servicios[0])
print("Último puerto:", puertos[-1])
```

3. Crear listas vacías y practicar append(), insert() y extend(). Cada método modifica la lista de una forma distinta.

```
lista_vacia_1 = []
lista_vacia_2 = list()
print(lista_vacia_1, lista_vacia_2)

servicios = ["ssh", "nginx"]
servicios.append("postgresql")
servicios.insert(1, "firewalld")
servicios.extend(["chronyd", "rsyslog"])
print(servicios)
```

4. Modificar y eliminar elementos. Revisar la diferencia entre cambiar por índice, borrar con del, eliminar por valor con remove() y extraer con pop().

```
servicios = ["ssh", "nginx", "postgresql"]
print("Lista inicial:", servicios)

servicios[1] = "httpd"
del servicios[0]
servicios.remove("postgresql")
servicios.append("rsyslog")
ultimo = servicios.pop()

print("Lista final:", servicios)
print("Elemento extraído:", ultimo)
```

5. Aplicar slicing para obtener subconjuntos de una lista. Recordar que el índice final no se incluye.

```
hosts = ["srv01", "srv02", "srv03", "srv04", "srv05"]
print(hosts[0:3])
print(hosts[:2])
print(hosts[2:])
print(hosts[:2])
print(hosts[:-1])
```

6. Recorrer listas directamente y con enumerate(). enumerate() es útil cuando se necesita posición y valor a la vez.

```
servicios = ["ssh", "nginx", "postgresql"]

for servicio in servicios:
    print("Servicio:", servicio)

for indice, servicio in enumerate(servicios, start=1):
    print(f"{indice}. {servicio}")

print("enumerate convertido a lista:", list(enumerate(servicios)))
```

7. Comprobar pertenencia, buscar posiciones con index() y contar apariciones con count().

```
servicios = ["ssh", "nginx", "ssh", "postgresql"]
print("ssh" in servicios)
print("firewalld" not in servicios)

if "nginx" in servicios:
    print("Índice de nginx:", servicios.index("nginx"))

print("Apariciones de ssh:", servicios.count("ssh"))
```

8. Ordenar y copiar listas. sorted() devuelve una lista nueva; sort() modifica la lista original; copy() crea una copia superficial.

```
servicios = ["nginx", "ssh", "postgresql", "api"]
print(sorted(servicios))
```

```
print(servicios)

servicios.sort(key=len)
print(servicios)

copia = servicios.copy()
copia.append("backup")
print(servicios)
print(copia)
```

9. Revisar la copia superficial con elementos mutables. La lista externa se copia, pero las listas internas siguen compartidas.

```
original = [{"srv01", "OK"}, {"srv02", "FALLO"}]
copia = original.copy()

copia[0][1] = "REVISAR"

print("Original:", original)
print("Copia:", copia)
```

10. Ejecutar el script completo.

```
python 01_listas.py
```

Resultado esperado: el alumno debe poder crear, recorrer, modificar, ordenar y copiar listas, y reconocer los efectos de una copia superficial.

Laboratorio 2. Listas multidimensionales

1. Abrir el fichero 02_listas_multidimensionales.py. Revisar una lista de listas como tabla sencilla.

```
estado_hosts = [
    ["srv01", "OK", 22],
    ["srv02", "FALLO", 80],
    ["srv03", "OK", 443],
]

print(estado_hosts[0][0])
print(estado_hosts[1][1])
print(estado_hosts[2][2])
```

2. Recorrer filas completas y asignar cada posición a una variable con nombre para mejorar la lectura.

```
for fila in estado_hosts:
    host = fila[0]
    estado = fila[1]
    puerto = fila[2]
    print(f"{host} -> {estado}:{puerto}")
```

3. Recorrer la matriz mediante índices de fila y columna. Este patrón es útil para entender matrices, aunque puede ser menos legible.

```
for indice_fila in range(len(estado_hosts)):
    print("Fila:", indice_fila)

    for indice_columna in range(len(estado_hosts[indice_fila])):
        print(" Valor:", estado_hosts[indice_fila][indice_columna])
```

4. Revisar una matriz irregular y usar slicing para separar el host de sus servicios.

```
servicios_por_host = [
    ["srv01", "ssh", "nginx"],
    ["srv02", "ssh"],
    ["srv03", "ssh", "postgresql", "backup"],
]

for fila in servicios_por_host:
    host = fila[0]
    servicios = fila[1:]
    print(f"{host} tiene servicios: {servicios}")
```

5. Comparar con una alternativa más legible usando diccionarios cuando los campos tienen nombre.

```
inventario = [
    {"host": "srv01", "estado": "OK", "puerto": 22},
    {"host": "srv02", "estado": "FALLO", "puerto": 80},
    {"host": "srv03", "estado": "OK", "puerto": 443},
]

for equipo in inventario:
    print(f"{equipo['host']} -> {equipo['estado']}:{equipo['puerto']}")
```

6. Ejecutar el script completo.

```
python 02_listas_multidimensionales.py
```

Resultado esperado: el alumno debe entender el acceso con [fila][columna] y cuándo un diccionario resulta más claro que una lista multidimensional.

Laboratorio 3. Comprensión de listas

1. Abrir el fichero 03_comprension_listas.py. Comparar una transformación con bucle tradicional y con comprensión de listas.

```
servicios = ["SSH ", " NGINX ", " postgresql "]
normalizados = []

for servicio in servicios:
    normalizados.append(servicio.strip().lower())
```

```
print(normalizados)

normalizados = [servicio.strip().lower() for servicio in servicios]
print(normalizados)
```

2. Filtrar datos con una condición dentro de la comprensión.

```
puertos = [22, 80, 443, 8080, 3306]
puertos_web = [puerto for puerto in puertos if puerto in (80, 443, 8080)]
print("Puertos web:", puertos_web)
```

3. Transformar y filtrar al mismo tiempo. En el ejemplo se extraen servicios desde líneas de log con error.

```
logs = ["OK ssh", "ERROR nginx", "ERROR api", "OK backup"]
servicios_error = [linea.split()[1] for linea in logs if linea.startswith("ERROR")]
print("Servicios con error:", servicios_error)
```

4. Aplicar comprensión de listas sobre una lista de objetos.

```
class Servicio:
    """Representa un servicio con nombre y estado."""

    def __init__(self, nombre, activo):
        self.nombre = nombre
        self.activo = activo

servicios = [
    Servicio("ssh", True),
    Servicio("api", False),
    Servicio("backup", True),
]

activos = [servicio.nombre for servicio in servicios if servicio.activo]
detenidos = [servicio.nombre for servicio in servicios if not servicio.activo]

print("Activos:", activos)
print("Detenidos:", detenidos)
```

5. Revisar el criterio práctico: si la comprensión deja de ser clara, usar un bucle tradicional.

```
resultado = []

for linea in logs:
    if linea.startswith("ERROR"):
        partes = linea.split()
        resultado.append(partes[1].upper())

print(resultado)
```

6. Ejecutar el script completo.

```
python 03_comprension_listas.py
```

Resultado esperado: el alumno debe crear listas por comprensión para transformaciones sencillas y reconocer cuándo un bucle tradicional es más legible.

Laboratorio 4. Tuplas

1. Abrir el fichero 04_tuplas.py. Crear una tupla y acceder a sus posiciones.

```
endpoint = ("srv-web-01", "10.0.0.20", 443)

print(endpoint)
print(type(endpoint))
print("Host:", endpoint[0])
print("IP:", endpoint[1])
print("Puerto:", endpoint[2])
```

2. Crear una tupla vacía y una tupla de un solo elemento. Recordar que la coma es obligatoria en la tupla de un elemento.

```
tupla_vacia_1 = ()
tupla_vacia_2 = tuple()
print(tupla_vacia_1, tupla_vacia_2)

solo_uno = ("ssh",)
no_es_tupla = ("ssh")

print(solo_uno, type(solo_uno))
print(no_es_tupla, type(no_es_tupla))
```

3. Desempaquetar una tupla en variables separadas.

```
host, ip, puerto = endpoint

print("Host:", host)
print("IP:", ip)
print("Puerto:", puerto)
```

4. Recorrer tuplas y practicar len(), in, slicing, count() e index().

```
puertos_estandar = (22, 80, 443, 5432)

print("Longitud:", len(puertos_estandar))
print("¿443 está permitido?", 443 in puertos_estandar)
print("Primeros dos puertos:", puertos_estandar[:2])

for puerto in puertos_estandar:
```

```
print(f"Puerto permitido: {puerto}")

puertos_repetidos = (22, 80, 443, 80, 8080)
print("Apariciones de 80:", puertos_repetidos.count(80))
print("Índice de 443:", puertos_repetidos.index(443))
```

5. Revisar la inmutabilidad y convertir temporalmente entre tupla y lista cuando sea necesario modificar datos.

```
# endpoint[2] = 8443 # TypeError si se descomenta

endpoint_lista = list(endpoint)
endpoint_lista[2] = 8443
endpoint_modificado = tuple(endpoint_lista)

print("Original:", endpoint)
print("Modificado:", endpoint_modificado)
```

6. Generar una tupla a partir de un iterable usando tuple(). Una expresión entre paréntesis por sí sola crea un generador.

```
puertos = [22, 80, 443, 8080]

generador = (puerto for puerto in puertos if puerto >= 100)
print("Tipo del generador:", type(generador))

puertos_altos = tuple(puerto for puerto in puertos if puerto >= 100)
print("Tupla generada:", puertos_altos)
```

7. Ejecutar el script completo.

```
python 04_tuplas.py
```

Resultado esperado: el alumno debe identificar las tuplas como secuencias ordenadas e inmutables, y practicar desempaqueado y conversión.

Laboratorio 5. Conjuntos

1. Abrir el fichero 05_conjuntos.py. Crear conjuntos y comprobar que eliminan duplicados.

```
etiquetas = {"prod", "linux", "prod", "web"}

print(etiquetas)
print(type(etiquetas))
print("¿prod está en etiquetas?", "prod" in etiquetas)

conjunto_vacio = set()
diccionario_vacio = {}
print(type(conjunto_vacio))
print(type(diccionario_vacio))
```

2. Añadir y eliminar elementos. Comparar discard() y remove().

```
roles = {"web", "db", "nosql", "app"}

roles.add("cache")
roles.add("web")
print("Tras add:", roles)

roles.discard("db")
roles.discard("no_existe")
print("Tras discard:", roles)

roles.remove("web")
print("Tras remove:", roles)

# roles.remove("backup") # KeyError si se descomenta
```

3. Deduplicar una lista convirtiéndola en conjunto. Recordar que no debemos asumir el orden original.

```
hosts = ["srv01", "srv02", "srv01", "srv03", "srv02"]

unicos = set(hosts)
print("Como set:", unicos)

hosts_unicos = list(unicos)
print("De nuevo como lista:", hosts_unicos)
```

4. Aplicar operaciones de conjuntos: intersección, unión, diferencia y diferencia simétrica.

```
permitidos = {22, 80, 443}
detectados = {22, 8080, 443, 3306}

print("Comunes:", permitidos & detectados)
print("Unión:", permitidos | detectados)
print("Detectados no permitidos:", detectados - permitidos)
print("Diferencias no comunes:", permitidos ^ detectados)
```

5. Usar métodos equivalentes y operaciones que modifican el conjunto original.

```
actuales = {"ssh", "nginx", "postgresql"}
requeridos = {"ssh", "nginx", "backup"}

faltan = requeridos - actuales
sobran = actuales - requeridos

print("Servicios que faltan:", faltan)
print("Servicios no requeridos:", sobran)

actuales.update(faltan)
print("Tras update:", actuales)
```


6. Crear conjuntos por comprensión para obtener valores únicos transformados o filtrados.

```
logs = ["ERROR ssh", "OK nginx", "ERROR api", "ERROR ssh"]
servicios_error = {linea.split()[1] for linea in logs if linea.startswith("ERROR")}
print("Servicios únicos con error:", servicios_error)

roles = ["WEB", "web", "Db", "db"]
roles_normalizados = {rol.lower() for rol in roles}
print("Roles normalizados únicos:", roles_normalizados)
```

7. Ejecutar el script completo.

```
python 05_conjuntos.py
```

Resultado esperado: el alumno debe usar conjuntos para pertenencia, deduplicación y comparación entre grupos de datos.

Laboratorio 6. Diccionarios

1. Abrir el fichero 06_diccionarios.py. Crear un diccionario y acceder a sus valores por clave.

```
servidor = {
    "hostname": "srv-web-01",
    "ip": "10.0.0.20",
    "activo": True,
}

print(servidor)
print(type(servidor))
print("Hostname:", servidor["hostname"])
print("IP:", servidor["ip"])
```

2. Practicar acceso seguro con get(). get() evita KeyError y permite indicar un valor por defecto.

```
servidor = {"hostname": "srv01", "ip": "10.0.0.20"}

print(servidor["hostname"])
print("Ubicación:", servidor.get("ubicacion"))
print("Ubicación con defecto:", servidor.get("ubicacion", "sin asignar"))

if servidor.get("ip"):
    print(f"IP detectada: {servidor['ip']}")

# print(servidor["ubicacion"]) # KeyError si se descomenta
```

3. Añadir, modificar y eliminar datos de un diccionario.

```
servicio = {"nombre": "nginx", "puerto": 80}
print("Inicial:", servicio)
```

```
servicio["activo"] = True
servicio["puerto"] = 443
print("Tras añadir/modificar:", servicio)

servicio.update({"protocolo": "tcp", "seguro": True})
print("Tras update:", servicio)

puerto = servicio.pop("puerto")
print("Tras pop:", servicio)
print("Puerto extraído:", puerto)

del servicio["seguro"]
print("Tras del:", servicio)
```

4. Recorrer claves, valores e items. `items()` suele ser la opción más clara cuando necesitamos clave y valor.

```
servicio = {
    "nombre": "nginx",
    "puerto": 443,
    "activo": True,
}

for clave in servicio:
    print(clave, servicio[clave])

for clave in servicio.keys():
    print("Clave:", clave)

for valor in servicio.values():
    print("Valor:", valor)

for clave, valor in servicio.items():
    print(f"{clave}: {valor}")
```

5. Trabajar con diccionarios anidados para representar inventarios con información estructurada.

```
inventario = {
    "srv-web-01": {
        "ip": "10.0.0.20",
        "servicios": ["ssh", "nginx"],
        "entorno": "prod",
    },
    "srv-db-01": {
        "ip": "10.0.0.30",
        "servicios": ["ssh", "postgresql"],
        "entorno": "prod",
    },
}

print("IP de srv-web-01:", inventario["srv-web-01"]["ip"])
```

```
for host, datos in inventario.items():
    print(f"{host} -> {datos['ip']} -> {datos['servicios']}")
```

6. Almacenar objetos dentro de un diccionario para recuperarlos rápidamente por clave.

```
class Servicio:
    """Representa un servicio con nombre y estado."""

    def __init__(self, nombre, activo):
        self.nombre = nombre
        self.activo = activo

    def comprobar(self):
        if self.activo:
            return "OK"
        return "DETENIDO"

servicios = {
    "ssh": Servicio("ssh", True),
    "api": Servicio("api", False),
}

print("ssh:", servicios["ssh"].comprobar())
print("api:", servicios["api"].comprobar())
```

7. Crear diccionarios por comprensión y construir un conteo simple de estados.

```
servicios = ["ssh", "nginx", "postgresql"]
puertos = [22, 80, 5432]

mapa_puertos = {servicios[i]: puertos[i] for i in range(len(servicios))}
print("Mapa de puertos:", mapa_puertos)

resultados = ["OK", "FALLO", "OK", "REVISAR", "FALLO", "OK"]
conteo = {}

for estado in resultados:
    if estado not in conteo:
        conteo[estado] = 0
    conteo[estado] += 1

print("Conteo de estados:", conteo)
```

8. Ejecutar el script completo.

```
python 06_diccionarios.py
```

Resultado esperado: el alumno debe poder crear diccionarios, acceder de forma segura, modificarlos, recorrerlos y combinarlos con listas u objetos.

Laboratorio 7. Ejemplo integrado con varias colecciones

1. Abrir el fichero 07_ejemplo_integrado_colecciones.py. Revisar la clase Servicio usada por el ejemplo.

```
class Servicio:
    """Representa un servicio instalado en un servidor."""

    def __init__(self, nombre, puerto, activo):
        self.nombre = nombre
        self.puerto = puerto
        self.activo = activo

    def comprobar(self):
        estado = "OK" if self.activo else "DETENIDO"
        return f"{self.nombre}:{self.puerto} -> {estado}"

    def __str__(self):
        return self.comprobar()
```

2. Crear una lista de objetos Servicio y recorrerla.

```
servicios_srv_web = [
    Servicio("ssh", 22, True),
    Servicio("nginx", 443, True),
    Servicio("api", 8080, False),
]

for servicio in servicios_srv_web:
    print(servicio)
```

3. Usar una tupla para representar un endpoint fijo: host, IP y entorno.

```
endpoint_web = ("srv-web-01", "10.0.0.20", "prod")

host, ip, entorno = endpoint_web
print(f"Endpoint: {host} -> {ip} -> {entorno}")
```

4. Usar conjuntos para etiquetas únicas y para detectar puertos no permitidos.

```
etiquetas = {"prod", "linux", "web", "prod"}
puertos_permitidos = {22, 80, 443}
puertos_detectados = {servicio.puerto for servicio in servicios_srv_web}

print("Etiquetas únicas:", etiquetas)
print("Puertos detectados:", puertos_detectados)
print("Puertos no permitidos:", puertos_detectados - puertos_permitidos)
```

5. Crear un diccionario principal de inventario que combine host, IP, entorno, etiquetas y servicios.

```
inventario = {
    host: {
```

```
        "ip": ip,
        "entorno": entorno,
        "etiquetas": etiquetas,
        "servicios": servicios_srv_web,
    }
}

print(inventario)
```

6. Recorrer el inventario y mostrar un resumen legible.

```
for nombre_host, datos in inventario.items():
    print(f"Servidor: {nombre_host}")
    print(f"IP: {datos['ip']}")
    print(f"Entorno: {datos['entorno']}")
    print(f"Etiquetas: {datos['etiquetas']}")
    print("Servicios:")

    for servicio in datos["servicios"]:
        print(" -", servicio.comprobar())
```

7. Generar reportes con comprensiones y calcular el resumen final.

```
servicios_activos = [
    servicio.nombre
    for servicio in servicios_srv_web
    if servicio.activo
]

mapa_puertos = {
    servicio.nombre: servicio.puerto
    for servicio in servicios_srv_web
}

servicios_por_estado = {
    servicio.nombre: ("OK" if servicio.activo else "DETENIDO")
    for servicio in servicios_srv_web
}

print("Servicios activos:", servicios_activos)
print("Mapa de puertos:", mapa_puertos)
print("Estado por servicio:", servicios_por_estado)
```

8. Ejecutar el script completo.

```
python 07_ejemplo_integrado_colecciones.py
```

Resultado esperado: el alumno debe ver un ejemplo coherente donde listas, tuplas, conjuntos, diccionarios y objetos trabajan juntos para construir un inventario sencillo.

Ejercicios propuestos

1. Crear un script `src/servicios_basicos.py` con una lista de servicios. Añadir dos servicios, eliminar uno, ordenar la lista y mostrarla numerada con `enumerate()`.
2. Crear un script `src/puertos_estandar.py` con una tupla de puertos estándar. Desempaquetar algunos valores y comprobar si 443 pertenece a la tupla.
3. Crear un script `src/deduplica_hosts.py` con una lista de hostnames repetidos. Convertirla a conjunto para eliminar duplicados y volver a mostrar los valores.
4. Crear un script `src/inventario_simple.py` con un diccionario que represente un servidor. Debe incluir hostname, IP, entorno, activo y lista de servicios.
5. Crear un script `src/puertos_no_permitidos.py` que compare dos conjuntos: puertos permitidos y puertos detectados. Mostrar los puertos detectados que no están permitidos.
6. Crear un script `src/reporte_servicios.py` con una lista de diccionarios de servicios. Usar una comprensión de listas para obtener solo los servicios activos.
7. Crear un script `src/inventario_integrado.py` que combine una tupla de endpoint, un conjunto de etiquetas, una lista de servicios y un diccionario final de inventario.

Guía de resolución de problemas

Problema	Diagnóstico	Solución
No aparece (Curso_Python) en el prompt	El entorno virtual no está activado.	Ejecutar <code>source .venv/bin/activate</code> desde <code>~/Curso_Python/Tema05</code> .
El script no se encuentra	La terminal no está situada en el directorio del tema.	Ejecutar <code>cd ~/Curso_Python/Tema05</code> y comprobar <code>ls -la</code> .
<code>IndexError</code> al acceder a una lista o tupla	El índice solicitado no existe.	Comprobar la longitud con <code>len()</code> y recordar que el primer índice es 0.
<code>KeyError</code> al acceder a un diccionario	La clave no existe en el diccionario.	Comprobar la clave con <code>in</code> o usar <code>get()</code> con un valor por defecto.
<code>TypeError</code> al acceder a un conjunto por índice	Los conjuntos no admiten acceso posicional.	Recorrer el conjunto con <code>for</code> o convertirlo a lista si realmente se necesita una secuencia.
<code>ValueError</code> con <code>remove()</code> o <code>index()</code>	El valor no existe en la colección.	Comprobar primero con <code>in</code> o usar <code>discard()</code> en conjuntos cuando no se conoce si el elemento existe.
La copia modifica también la lista original	Se ha hecho una copia superficial de una colección con	Revisar si se necesita una copia profunda. En este tema basta

	objetos mutables internos.	con reconocer que se comparten referencias internas.
El resultado de un set aparece en otro orden	Los conjuntos no deben usarse cuando el orden sea parte del problema.	Usar una lista o una tupla si el orden de presentación o acceso es importante.